

SISTEMA DE MONITORAMENTO DE RISCO EM TEMPO REAL

**UTILIZANDO VISÃO COMPUTACIONAL E COMPUTAÇÃO DE
BORDA**

Relatório Técnico

Londrina, Janeiro de 2025

1. Introdução

O avanço das técnicas de visão computacional e aprendizado profundo tem viabilizado o desenvolvimento de sistemas capazes de interpretar informações visuais de forma automática e em tempo real. Em ambientes industriais, essas tecnologias são amplamente empregadas para aumentar a segurança operacional, reduzir riscos e auxiliar na tomada de decisões relacionadas à prevenção de acidentes.

Nesse contexto, o monitoramento contínuo de atividades potencialmente perigosas representa um desafio relevante, especialmente em cenários nos quais a supervisão humana constante é inviável, limitada por recursos ou sujeita a falhas operacionais. Sistemas automatizados de detecção de objetos permitem identificar situações de risco de forma proativa, contribuindo para a mitigação de acidentes e para o atendimento a requisitos de segurança e conformidade.

Este relatório apresenta a documentação técnica do Edge Risk Monitor, um sistema de monitoramento de risco baseado em visão computacional, projetado para realizar inferência em tempo real utilizando computação de borda (edge computing). A solução executa localmente a detecção de objetos de risco, aplica critérios temporais e estatísticos para confirmação de eventos e gera evidências visuais associadas às ocorrências detectadas.

Para fins de desenvolvimento e validação do fluxo completo de monitoramento, foi adotado como objeto de risco um mouse de computador, utilizado como elemento representativo de um item perigoso. Essa abordagem permite avaliar a arquitetura, a lógica de decisão e a integração com sistemas externos sem exposição a riscos reais, mantendo o foco nos aspectos técnicos e metodológicos da solução.

2. Objetivos

2.1 Objetivo Geral

Desenvolver um sistema de monitoramento de risco em tempo real, baseado em visão computacional, capaz de identificar automaticamente a presença de um objeto de risco em um fluxo contínuo de vídeo e registrar eventos associados às detecções confirmadas.

2.2 Objetivos Específicos

- Implementar um sistema de captura contínua de imagens por meio de webcam.
- Realizar inferência local em tempo real utilizando um modelo de detecção de objetos.
- Aplicar um mecanismo de decisão temporal (debounce) para redução de falsos positivos.
- Gerar evidências visuais associadas às detecções confirmadas.
- Enviar eventos estruturados para um backend desacoplado via API HTTP.
- Garantir organização, modularidade e rastreabilidade do código-fonte.

3. Visão Geral do Sistema

A arquitetura do Edge Risk Monitor foi projetada com foco em execução local, baixa latência, desacoplamento de responsabilidades e facilidade de integração com sistemas externos. O sistema adota o paradigma de computação de borda (edge computing), no qual toda a lógica de captura, inferência e decisão de risco ocorre localmente no dispositivo de monitoramento.

A comunicação com sistemas externos é realizada exclusivamente para o envio de eventos e evidências, garantindo que a operação do monitoramento não dependa da disponibilidade contínua do backend.

3.1 Visão Geral da Arquitetura

A solução é composta por dois blocos principais:

- **Camada Edge (Local)**
Responsável pela captura de vídeo, execução da inferência, decisão de risco, geração de evidências e orquestração do fluxo de monitoramento.
- **Camada Backend (Remota)**
Responsável pela recepção, persistência e posterior análise dos eventos enviados pelo edge, por meio de uma API HTTP desacoplada.

Toda a lógica crítica do sistema está concentrada na camada edge, enquanto o backend atua como consumidor de eventos, não interferindo no processo decisório local.

3.2 Componentes da Camada Edge

A camada edge é composta pelos seguintes componentes principais:

- **Webcam**
Responsável pela aquisição contínua de frames de vídeo a partir do dispositivo físico de captura, utilizando a biblioteca OpenCV.
- **Detector**
Encapsula a lógica de inferência baseada em um modelo de detecção de objetos (YOLO), realizando a identificação do objeto de risco em cada frame capturado.
- **Mecanismo de Decisão Temporal**
Implementa uma lógica baseada em janela deslizante de confiança, utilizada para confirmar ou limpar o estado de risco com base em critérios estatísticos, reduzindo a ocorrência de falsos positivos.
- **Evidence Saver**
Responsável pelo armazenamento local de evidências visuais associadas às detecções confirmadas, garantindo rastreabilidade das ocorrências.

- **Event Sender**

Encapsula a comunicação HTTP com o backend externo, realizando o envio de payload estruturado e evidências visuais por meio de requisições multipart/form-data.

- **Main (Orquestrador)**

Atua como ponto de entrada do sistema, coordenando a inicialização dos componentes, o loop principal de execução e o fluxo de dados entre os módulos, sem implementar regras de negócio específicas.

3.3 Camada de Integração com Backend

A integração com o backend ocorre de forma assíncrona e desacoplada, sendo acionada apenas quando um evento de risco é confirmado. O edge envia:

- Metadados estruturados do evento (identificação do dispositivo, classe detectada e timestamp).
- Arquivo de evidência visual associado à detecção.

A indisponibilidade temporária do backend não compromete a execução da inferência local, reforçando o princípio de autonomia da camada edge.

3.4 Princípios Arquiteturais Adotados

A arquitetura do sistema segue os seguintes princípios:

- **Separação de Responsabilidades**
Cada módulo possui escopo bem definido e isolado.
- **Baixo Acoplamento**
A comunicação entre componentes é realizada por interfaces simples e explícitas.
- **Execução Determinística**
A decisão de risco é baseada em critérios objetivos e reproduzíveis.
- **Escalabilidade Funcional**
A arquitetura permite a substituição do modelo de detecção, da classe monitorada ou do backend sem alterações estruturais significativas.
- **Robustez Operacional**
O sistema mantém sua operação mesmo na ausência temporária de conectividade com serviços externos.

4. Arquitetura do Sistema

O fluxo de execução do Edge Risk Monitor é determinístico e sequencial, sendo coordenado pelo módulo main.py, que atua exclusivamente como orquestrador do pipeline de monitoramento. Este módulo não implementa regras de negócio, mas integra e coordena os componentes previamente definidos.

O ciclo de execução pode ser dividido em quatro etapas principais: inicialização, execução contínua, decisão de risco e encerramento controlado.

4.1 Inicialização do Sistema

Ao iniciar a aplicação, o sistema executa as seguintes ações:

- **Inicialização do sistema de logging**
O mecanismo de logging global é configurado, permitindo o registro estruturado de eventos, falhas e estados do sistema.
- **Inicialização da webcam**
O dispositivo de captura de vídeo é instanciado e configurado com os parâmetros definidos em arquivo de configuração, como índice da câmera e resolução dos frames.
- **Carregamento do modelo de detecção**
O modelo de detecção de objetos baseado em YOLO é carregado em memória, tornando-se disponível para inferência em tempo real.
- **Inicialização dos componentes auxiliares**
São instanciados os módulos responsáveis pelo salvamento de evidências visuais e pelo envio de eventos ao backend externo.
- **Autenticação na API externa**
O sistema realiza a autenticação no backend, obtendo um token de acesso utilizado nas requisições subsequentes de envio de eventos.
- **Obtenção do identificador do dispositivo**
Um identificador único do dispositivo edge é obtido, permitindo a rastreabilidade dos eventos enviados.

Após a conclusão dessas etapas, o sistema encontra-se apto para iniciar o monitoramento contínuo.

4.2 Loop Principal de Monitoramento

O núcleo da aplicação é composto por um loop contínuo, executado enquanto o sistema estiver ativo. A cada iteração, as seguintes operações são realizadas:

- **Captura de frame**
Um frame é capturado a partir da webcam. Caso a captura falhe, o sistema prossegue para a próxima iteração sem interromper a execução.
- **Execução da inferência**
O frame capturado é submetido ao módulo de detecção, que avalia a presença do objeto de risco e retorna, quando aplicável, informações como classe detectada, confiança e bounding box.

- **Persistência visual temporária**

Quando uma detecção ocorre, o sistema mantém a exibição do bounding box por um número configurável de frames, mesmo na ausência momentânea de novas detecções, garantindo estabilidade visual.

4.3 Decisão de Risco

A decisão de risco não é baseada em uma detecção isolada, mas em um mecanismo temporal:

- **Atualização da janela de confiança**

A cada frame, o valor de confiança da detecção é adicionado a uma janela deslizante de tamanho fixo.

- **Cálculo da média de confiança**

A média dos valores presentes na janela é utilizada como critério estatístico para decisão.

- **Confirmação do risco**

Quando a média de confiança ultrapassa um limiar superior configurado, o sistema considera o risco confirmado e altera seu estado interno.

- **Limpeza do estado de risco**

Quando a média de confiança cai abaixo de um limiar inferior, o estado de risco é removido, permitindo novas detecções futuras.

Esse mecanismo reduz significativamente a ocorrência de falsos positivos causados por detecções esporádicas ou ruídos visuais.

4.4 Geração de Evidências e Envio de Eventos

Uma vez confirmado o estado de risco, o sistema executa as seguintes ações:

- **Geração de evidência visual**

O frame correspondente à detecção é salvo localmente como evidência, com nome baseado em timestamp para garantir unicidade e rastreabilidade.

- **Construção do payload do evento**

São agregadas informações estruturadas, como identificador do dispositivo, classe detectada e data/hora da ocorrência.

- **Envio do evento ao backend**

O payload e a evidência visual são enviados ao backend externo por meio de uma requisição HTTP POST no formato multipart/form-data.

- **Controle de envio único**

O sistema garante que apenas um evento seja enviado por ativação de risco, evitando duplicações até que o estado de risco seja limpo.

4.5 Encerramento Controlado

O sistema pode ser encerrado manualmente pelo operador. Ao final da execução:

- Os recursos da webcam são liberados de forma controlada.
- As janelas de visualização são fechadas.
- O encerramento do pipeline é registrado nos logs.

Esse procedimento garante que não haja vazamento de recursos ou inconsistências no estado do sistema.

5. Metodologia de Desenvolvimento

5.1 Captura de Vídeo

A captura de imagens é realizada por meio da biblioteca OpenCV, utilizando uma webcam conectada ao sistema. O módulo responsável por essa funcionalidade abstrai completamente o acesso ao dispositivo físico de captura, fornecendo frames no formato BGR, compatível com o pipeline de inferência adotado.

Esse isolamento arquitetural garante que a lógica de aquisição de vídeo permaneça desacoplada das etapas de inferência, decisão de risco e integração externa, facilitando a manutenção do código, a realização de testes e a eventual substituição do dispositivo de captura sem impacto nas demais camadas do sistema.

5.2 Inferência em Tempo Real

A detecção do objeto de risco é realizada por meio de um modelo de detecção de objetos baseado na arquitetura YOLO, previamente treinado. O modelo é carregado durante a inicialização do sistema e executa inferência frame a frame sobre o fluxo contínuo de vídeo.

Para cada frame processado, o modelo retorna possíveis bounding boxes, classes associadas e níveis de confiança. Apenas as detecções que atendem simultaneamente aos seguintes critérios são consideradas válidas:

- Correspondência com a classe de interesse definida em configuração.
- Confiança acima de um limiar mínimo estabelecido.
- Atendimento a filtros geométricos aplicados às bounding boxes, visando a redução de falsos positivos.

A partir dessas restrições, o sistema seleciona apenas a melhor evidência por frame, garantindo consistência e evitando múltiplas detecções concorrentes.

5.3 Decisão Temporal Baseada em Confiança

Para reduzir a ocorrência de falsos positivos decorrentes de ruídos ou detecções esporádicas, o sistema implementa um mecanismo de decisão temporal baseado em média de confiança em janela deslizante, em vez de simples contagem de frames consecutivos.

A cada iteração do loop principal, o valor de confiança da detecção (ou zero, na ausência de detecção) é inserido em uma janela temporal de tamanho fixo. A média dessa janela é então utilizada como critério para:

- Confirmação do estado de risco, quando a média de confiança ultrapassa um limiar superior.
- Limpeza do estado de risco, quando a média de confiança cai abaixo de um limiar inferior.

Essa abordagem introduz um mecanismo de histerese, evitando oscilações frequentes do estado de risco e proporcionando maior robustez frente a variações momentâneas da inferência.

5.4 Geração de Evidências

Uma vez confirmado o estado de risco, o sistema realiza a captura e o armazenamento de uma evidência visual correspondente ao evento. A evidência consiste em uma imagem do frame associado à detecção confirmada, armazenada localmente com nome baseado em timestamp.

Esse procedimento garante rastreabilidade temporal do evento e possibilita auditoria posterior, sem interferir no desempenho do pipeline de inferência em tempo real.

5.5 Envio de Eventos

Após a geração da evidência visual, o sistema envia um evento estruturado para um backend desacoplado por meio de uma requisição HTTP. O envio é realizado no formato multipart/form-data, contendo:

- Dados estruturados do evento (identificador do dispositivo, data/hora e classe detectada).
- Arquivo da evidência visual associada.

A responsabilidade pela validação semântica e persistência definitiva dos eventos é delegada ao backend, mantendo o sistema edge focado exclusivamente na detecção e despacho dos eventos.

6. Considerações Arquiteturais

O sistema foi projetado com base em princípios de modularidade e separação de responsabilidades, adotando uma arquitetura na qual cada componente desempenha uma função bem definida e isolada. Os módulos responsáveis por captura de vídeo, inferência, decisão de risco, geração de evidências e envio de eventos operam de forma independente, comunicando-se por meio de interfaces claras.

O arquivo `main.py` exerce exclusivamente o papel de orquestrador do fluxo de execução, coordenando a interação entre os componentes sem incorporar regras de negócio específicas. Essa abordagem favorece baixo acoplamento, alta coesão e facilidade de manutenção, além de permitir a evolução futura do sistema por meio da substituição ou

aprimoramento de módulos individuais, sem impacto significativo nas demais partes da aplicação.

7. Resultados Obtidos

Durante os testes realizados, o sistema demonstrou capacidade de detectar o objeto de risco em tempo real, confirmando corretamente os eventos por meio do mecanismo de decisão temporal baseado em média de confiança. O controle de estado implementado evitou múltiplos envios de eventos para uma mesma ocorrência, garantindo consistência e previsibilidade do comportamento do sistema.

As evidências visuais foram geradas e armazenadas conforme esperado, permitindo rastreabilidade temporal dos eventos detectados. Além disso, os eventos estruturados foram enviados com sucesso ao backend configurado, validando a integração entre o sistema edge e a API externa.

Os logs gerados possibilitaram o acompanhamento detalhado de todo o ciclo de vida da aplicação, desde a captura dos frames, passando pela inferência e decisão de risco, até o envio final dos eventos, contribuindo para diagnóstico, auditoria e análise do comportamento do sistema.

8. Limitações

Apesar de atender aos objetivos propostos, o sistema apresenta algumas limitações que devem ser consideradas em cenários de evolução ou uso em ambientes produtivos:

- O sistema oferece suporte a apenas uma câmera por instância, não contemplando, no estado atual, cenários com múltiplos pontos de monitoramento simultâneos.
- A detecção está restrita a um único tipo de objeto de risco, previamente definido durante o treinamento do modelo de detecção.
- O desempenho da inferência é dependente das condições de iluminação, enquadramento e posicionamento da câmera, fatores que podem influenciar diretamente a precisão do modelo.
- O envio de eventos ao backend não contempla, nesta implementação, mecanismos avançados de segurança, como criptografia de comunicação ou políticas de autenticação mais robustas, os quais podem ser incorporados em versões futuras conforme requisitos de produção.

9. Conclusão

O desenvolvimento do Edge Risk Monitor demonstrou a viabilidade de um sistema de monitoramento de risco baseado em visão computacional, operando em tempo real por meio de computação de borda. A arquitetura modular adotada contribuiu para a organização do código, clareza das responsabilidades e facilidade de integração entre os componentes do sistema.

A utilização de técnicas como decisão temporal baseada em média de confiança mostrou-se eficaz na redução de falsos positivos, aumentando a robustez do processo de detecção. A solução apresentada atende aos objetivos definidos e estabelece uma base sólida para a evolução de sistemas mais complexos de monitoramento de risco em ambientes industriais.